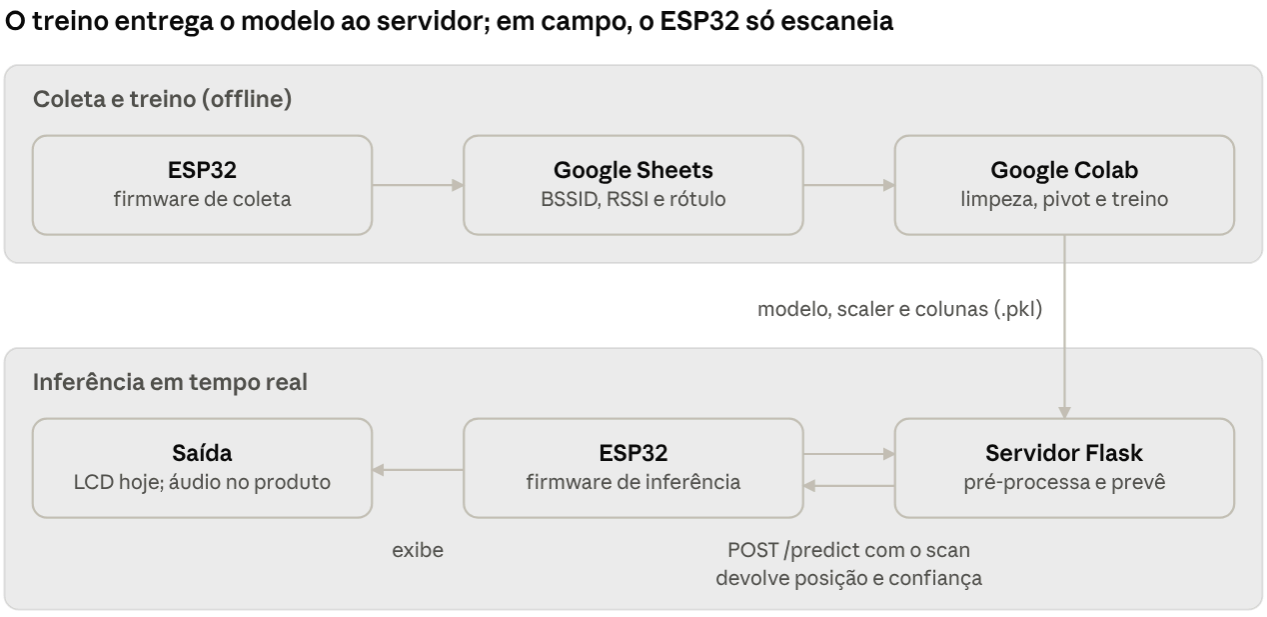
VidiWaze — Registro de avanços: protótipo físico e inferência em tempo real

Oct 19, 2026 · @Ruan

Arquitetura do sistema

O sistema passou a ter dois modos com firmwares separados, ligados apenas pelos arquivos do modelo treinado.



arquitetura do VidiWaze · coleta e inferência

Na coleta, o ESP32 grava os scans rotulados na planilha e o Colab treina o modelo. Em campo, o ESP32 só escaneia e envia; o servidor aplica o mesmo pré-processamento do treino e devolve a posição.

Protótipo físico

O protótipo passou a ser alimentado por bateria LiPo, para coletar e testar no corredor sem o notebook preso ao braço.

Componente	Função	Observação
ESP32 DevKit (WROOM-32U)	Scan Wi-Fi e comunicação	Alimentado pelo pino VIN (5 V)

Componente	Função	Observação
Bateria LiPo 3,7 V	Alimentação portátil	⩾ 1000 mAh, com proteção
Módulo TP4056 com proteção (DW01)	Carga da bateria via USB	Carrega e alimenta o circuito ao mesmo tempo
Módulo step-up MT3608	Eleva 3,2–4,2 V da bateria para 5,0 V	Trimpot ajustado com multímetro
Capacitor eletrolítico 470 µF	Absorve picos de corrente do rádio Wi-Fi	Evita o *brownout* visto no início do projeto
LCD 16×2 com adaptador I²C	Status durante o desenvolvimento	SDA = GPIO21, SCL = GPIO22
2 botões	Escolher e confirmar o ponto de coleta	GPIO14 e GPIO27, com INPUT_PULLUP

Cadeia de alimentação: USB → TP4056 → MT3608 (5,0 V) → VIN do ESP32. Com o USB conectado, o TP4056 carrega a bateria e mantém o circuito ligado; sem USB, a bateria assume.

A autonomia estimada é de 4 a 6 horas com 1000 mAh, para um consumo de 150 a 250 mA. É uma estimativa de projeto, ainda não medida.

Esquemático elétrico e correções

O esquemático em nível de módulos está pronto (PDF A4, Rev. A, 07/10/2026, arquivo esquematico_vidiwaze.pdf), com lista de componentes e notas de montagem. Ao desenhá-lo, três pontos do esboço anterior foram corrigidos:

Item	Esboço anterior	Correção	Motivo
Alimentação do LCD	3V3 do ESP32	5 V (trilho do MT3608)	O LCD 16×2 precisa de 5 V para contraste e backlight
Linhas I²C	Ligadas direto aos GPIOs	Level shifter (BSS138) ou pull-ups de 4,7 kΩ para 3V3	Os pull-ups do adaptador levariam 5 V aos GPIO21/22
B− do TP4056	Tratado como GND comum	Ligado direto ao polo negativo da bateria	O MOSFET de proteção fica entre B− e OUT−

Notas de montagem registradas no PDF:

- Ajustar o MT3608 para 5,0 V antes de ligar o ESP32.
- Não ligar o USB do ESP32 ao PC com a bateria alimentando o VIN, salvo se a placa tiver diodo de isolamento.
- O TP4056 não tem *power-path* completo: carregar com o ESP32 ligado funciona, mas o fim de carga pode ser impreciso.

Servidor de inferência (Flask)

O servidor de predição foi escrito e testado com um modelo sintético; ainda falta rodá-lo com o modelo real treinado nos dados do ESP32. É uma solução de prova de conceito: no produto, o modelo deve rodar embarcado ou em um dispositivo levado pelo usuário.

Do Colab saem três arquivos: o Random Forest treinado, o `MinMaxScaler` ajustado no treino e a lista de BSSIDs na ordem das colunas do treino. O scikit-learn do servidor precisa ter a mesma versão do Colab para o arquivo do modelo carregar corretamente.

A cada requisição `POST /predict`, o servidor:

1. Lê o JSON e converte os BSSIDs para maiúsculas, como no treino.
2. Monta o vetor na ordem das colunas do treino, com -100 dBm para os APs ausentes.
3. Ignora BSSIDs que o modelo não conhece e informa quantos foram ignorados.
4. Aplica o scaler e roda o modelo.
5. Devolve a posição, a confiança e as 3 posições mais prováveis.

O teste com o modelo sintético cobriu a rota `/health`, uma predição válida e quatro entradas inválidas (JSON malformado, nenhum BSSID conhecido, RSSI não numérico e scan vazio), todas recusadas com HTTP 400. O próximo teste é enviar scans reais da planilha e conferir se a posição devolvida bate com o rótulo.